Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №4
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Бинарное дерево поиска»

Выполнили:
студенты группы 25ВВВ4
Суслова А.В.
Саченкова О.А.

Приняли:
Юрова О.В.,
Деев М.В.

Пенза 2026

1. Цель

Изучить бинарное дерево поиска, условия его организации и функционирования.

2. Задания

Задание 1: реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.

Задание 2: реализовать функцию подсчёта числа вхождений заданного элемента в дерево.

Задание 3: изменить функцию добавления элементов для исключения добавления одинаковых символов.

Задание 4: оценить сложность процедуры поиска по значению в бинарном дереве.

3. Пояснительный текст к программам

Задания 1-2

В программе реализовано бинарное дерево поиска. Для хранения данных используется структура Node, содержащая целочисленное значение и указатели на левое и правое поддеревья. Память для каждого нового узла выделяется динамически. Построение дерева осуществляется функцией insert(). Если новое значение меньше значения текущего узла, оно добавляется в левое поддерево. Если значение больше или равно значению текущего узла, оно добавляется в правое поддерево (здесь допускаются повторения).

Для поиска заданного значения используется функция search(). Она сравнивает искомое значение со значением текущего узла и, в зависимости от результата сравнения, продолжает поиск в левом или правом поддереве. Если значение найдено, функция возвращает указатель на соответствующий узел.

Если достигнут конец ветви дерева, то есть указатель равен NULL, функция сообщает об отсутствии искомого значения.

Подсчёт количества вхождений выполняется функцией countOccurrences(). Она рекурсивно обходит оба поддерева, сравнивает значения узлов с искомым числом и суммирует количество совпадений.

Для отображения структуры дерева используется функция print_tree(), которая выводит его повёрнутым на 90 градусов. После завершения работы функция freeTree() освобождает динамически выделенную память.

Результат работы программы:

```
Enter -1 to finish building.

Enter a value: 12
Enter a value: 34
Enter a value: 56
Enter a value: 11
Enter a value: 111
Enter a value: 12
Enter a value: -34
Enter a value: 32
Enter a value: 12
Enter a value: 8
Enter a value: -1

Tree:
            111
        56
    34
            32
                12
        12
12
    11
            8
        -34

Enter a value to search for: 12
Value 12 was found.
Number of occurrences of 12: 3
```

Задание 3

Во второй программе изменена функция insert(), которая отвечает за добавление новых элементов. Функция проверяет три возможных случая: если новое значение меньше значения текущего узла, оно добавляется в левое поддерево; если больше — в правое; если значения равны, добавление не выполняется и выводится соответствующее сообщение.

Таким образом, повторяющиеся значения исключаются. Вследствие этого функция countOccurrences() может возвращать только 0, если заданное значение отсутствует в дереве, или 1, если оно присутствует.

Результат работы программы:

```
Enter -1 to finish building.

Enter a value: 21
Enter a value: 564
Enter a value: -12
Enter a value: 12
Enter a value: 0
Enter a value: 7
Enter a value: 34
Enter a value: 34
Value already exists: 34
Enter a value: 7
Value already exists: 7
Enter a value: 87
Enter a value: 22
Enter a value: -1

Tree:
    564
            87
        34
            22
21
        12
                7
            0
    -12

Enter a value to search for: 0
Value 0 was found.
Number of occurrences of 0: 1
```

Задание 4

Есть 3 варианта оценки сложности процедуры для функции поиска, они зависят от положения конкретного элемента в дереве.

Если элемент, который мы ищем, находится в корне, то сложность будет **O(1)**, так как было проведено всего одно сравнение.

Если дерево достаточно сбалансировано, при каждом переходе область поиска примерно уменьшается вдвое. Число сравнений вырастет логарифмически, и сложность будет **O(log n)**.

Если дерево не сбалансировано (есть длинная цепочка поддеревьев), то сложность будет **O(n)**, так как проверка пойдёт по всем узлам.

4. Псевдокод

Задание 1-2

Алгоритм добавления элемента

```
НАЧАЛО

    получить корень дерева и новое значение

    ЕСЛИ корень отсутствует
        создать новый узел
        вернуть его
    КОНЕЦ ЕСЛИ

    ЕСЛИ новое значение меньше значения узла
        рекурсивно добавить значение в левое поддерево
    ИНАЧЕ ЕСЛИ новое значение больше значения узла
        рекурсивно добавить значение в правое поддерево
    ИНАЧЕ
        рекурсивно добавить значение в правое поддерево
    КОНЕЦ ЕСЛИ

    вернуть корень дерева
```

КОНЕЦ

Алгоритм поиска

```
НАЧАЛО

    получить корень дерева и искомое значение

    ЕСЛИ корень отсутствует
        вернуть NULL
    КОНЕЦ ЕСЛИ

    ЕСЛИ значение узла равно искомому
        вернуть адрес узла
    КОНЕЦ ЕСЛИ

    ЕСЛИ искомое значение меньше значения узла
        искать в левом поддереве
    ИНАЧЕ
        искать в правом поддереве
    КОНЕЦ ЕСЛИ

КОНЕЦ
```

Алгоритм расчёта повторений

```
НАЧАЛО

    получить корень дерева и искомое значение

    ЕСЛИ корень отсутствует
        вернуть 0
    КОНЕЦ ЕСЛИ

    установить счётчик в 0

    ЕСЛИ значение текущего узла равно искомому
        увеличить счётчик на 1
    КОНЕЦ ЕСЛИ

    рекурсивно подсчитать вхождения в левом поддереве
    рекурсивно подсчитать вхождения в правом поддереве

    вернуть сумму трёх результатов

КОНЕЦ
```

Задание 3

Алгоритм добавления элемента

```
НАЧАЛО

    получить корень дерева и новое значение

    ЕСЛИ корень отсутствует
        создать новый узел
        вернуть его
    КОНЕЦ ЕСЛИ

    ЕСЛИ новое значение меньше значения узла
        рекурсивно добавить значение в левое поддерево
    ИНАЧЕ ЕСЛИ новое значение больше значения узла
        рекурсивно добавить значение в правое поддерево
    ИНАЧЕ
        сообщить, что значение уже существует
    КОНЕЦ ЕСЛИ

    вернуть корень дерева

КОНЕЦ
```

Вывод

Изучили бинарное дерево поиска, условия его организации и функционирования.